

```
In [ ]: ASSIGNMENT NO : 4
Build a simple Rule-based Classifier or Random Forest Model to evaluate password length, special characters, and dictionary words.
```

```
In [ ]: NAME : VISHAKHA VIKAS CHAVAN
ROLL NO : EN24107026
```

```
In [1]: import pandas as pd
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, classification_report, confusion_m
```

```
In [2]: df = pd.read_csv("passwords_dataset.csv")
```

```
In [3]: df
```

```
Out[3]:
```

	Password	Has Lowercase	Has Uppercase	Has Special Character	Length	Strength
0	<%r?.	True	False	True	5	Weak
1	l(d_l	True	False	True	5	Weak
2	+Z)kDTRYo:q{"(	True	True	True	15	Strong
3	gwcNB[oS5!n%OPJ	True	True	True	15	Strong
4	^vXjCCP6	True	True	True	8	Strong
...	...	...	...	...	...	...
9995	2h"H6wHY\$?tIG<	True	True	True	14	Strong
9996	.jF?Y7:_"mQ0?</	True	True	True	15	Strong
9997	KBcVGjvqBIH	True	True	False	11	Medium
9998	P9C[;Z	False	True	True	6	Medium
9999	_Cok}{^%/4>G%	True	True	True	13	Strong

10000 rows × 6 columns

```
In [4]: df.head()
```

```
Out[4]:
```

	Password	Has Lowercase	Has Uppercase	Has Special Character	Length	Strength
0	<%r?.	True	False	True	5	Weak
1	l(d_l	True	False	True	5	Weak
2	+Z)kDTRYo:q{"(	True	True	True	15	Strong
3	gwcNB[oS5!n%OPJ	True	True	True	15	Strong
4	^vXjCCP6	True	True	True	8	Strong

```
In [5]: df.tail()
```

	Password	Has Lowercase	Has Uppercase	Has Special Character	Length	Strength
9995	2h"H6wHY\$?tIG<	True	True	True	14	Strong
9996	.jF?Y7:_"mQ0?</	True	True	True	15	Strong
9997	KBcVGjvqBIH	True	True	False	11	Medium
9998	P9C[;Z	False	True	True	6	Medium
9999	_Cok][^%/4>G%	True	True	True	13	Strong

```
In [6]: df.isnull().sum()
```

```
Out[6]: Password          0
Has Lowercase          0
Has Uppercase          0
Has Special Character  0
Length                 0
Strength               0
dtype: int64
```

```
In [7]: df.columns
```

```
Out[7]: Index(['Password', 'Has Lowercase', 'Has Uppercase', 'Has Special Character',
              'Length', 'Strength'],
              dtype='object')
```

```
In [8]: X = df[["Has Lowercase", "Has Uppercase", "Has Special Character", "Length"]]
y = df["Strength"]
```

```
In [9]: #X = Password ke features
#y = Password ki strength
```

```
In [10]: X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.20,
                                                             random_state=42, stratify=y)
```

```
In [11]: model = RandomForestClassifier(n_estimators=100, random_state=42)
```

```
In [12]: #n_estimators=100 means in random forest there are 100 decision trees used.
```

```
In [13]: model.fit(X_train, y_train)
print("Random Forest Model Trained Successfully!")
```

Random Forest Model Trained Successfully!

```
In [14]: y_pred = model.predict(X_test)
```

```
print("Predictions:")
print(y_pred[:10])
```

```
Predictions:
['Strong' 'Strong' 'Medium' 'Medium' 'Strong' 'Strong' 'Strong' 'Strong'
 'Medium' 'Strong']
```

```
In [15]: accuracy = accuracy_score(y_test, y_pred)
```

```
print("MODEL PERFORMANCE")
print("Accuracy:", accuracy)
print("Accuracy (%)ate", accuracy * 100)
```

```
MODEL PERFORMANCE
Accuracy: 0.9995
Accuracy (%): 99.95
```

```
In [16]: print("Classification Report:")
print(classification_report(y_test, y_pred))
```

```
Classification Report:
              precision    recall  f1-score   support

   Medium         1.00        1.00        1.00        370
   Strong         1.00        1.00        1.00       1191
   Weak          1.00        1.00        1.00        439

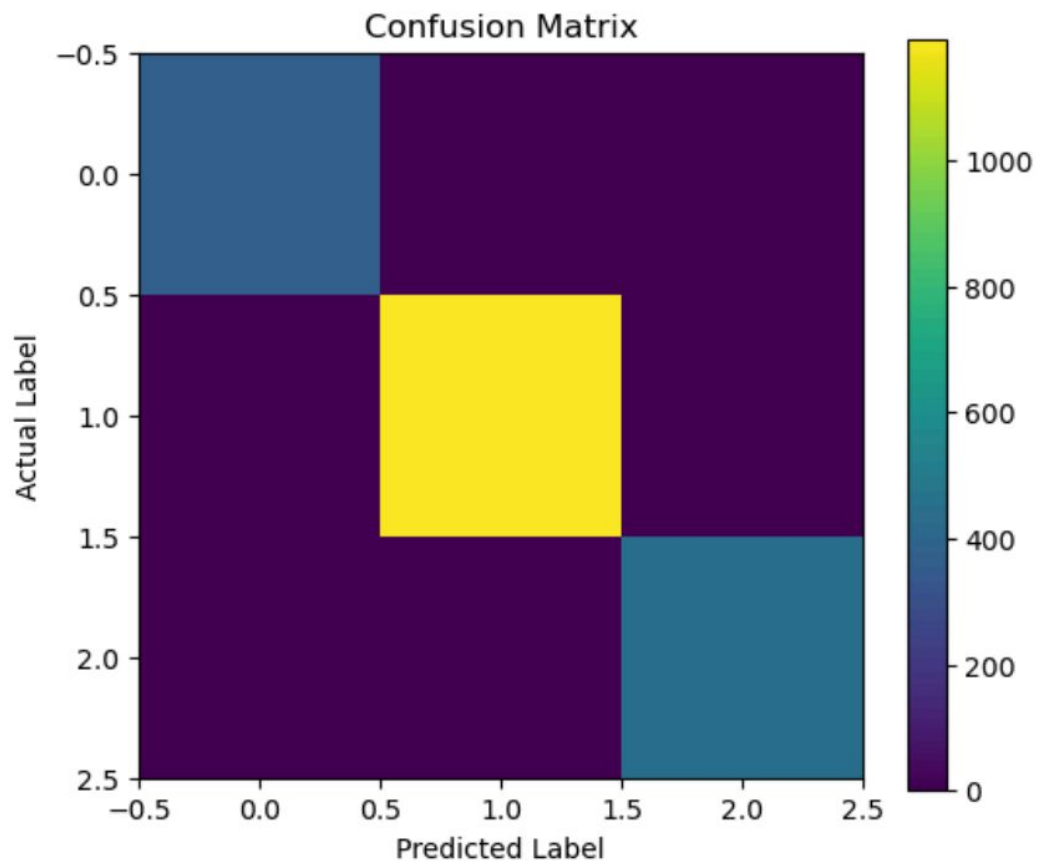
 accuracy                   1.00        2000
 macro avg         1.00        1.00        1.00        2000
 weighted avg      1.00        1.00        1.00        2000
```

```
In [17]: cm = confusion_matrix(y_test, y_pred)

print("Confusion Matrix:")
print(cm)
```

```
Confusion Matrix:
[[ 370   0   0]
 [   0 1191   0]
 [   0   1  438]]
```

```
In [18]: plt.figure(figsize=(6, 5))
plt.imshow(cm)
plt.xlabel("Predicted Label")
plt.ylabel("Actual Label")
plt.title("Confusion Matrix")
plt.colorbar()
plt.show()
```



```
In [19]: #1
new_password = [
    1,  # Has Lowercase
    1,  # Has Uppercase
    1,  # Has Special Character
    1   # Length
]

prediction = model.predict(new_password)

print("NEW PASSWORD PREDICTION")
print("Predicted Strength:", prediction[0])
```

```
NEW PASSWORD PREDICTION
Predicted Strength: Weak
```

```
/home/admin1/anaconda3/lib/python3.9/site-packages/sklearn/base.py:450: UserWarning: X does not have valid feature names, but RandomForestClassifier was fitted with feature names
  warnings.warn(
```

```
In [20]: #2
new_password = [[1, 1, 1, 12]]

prediction = model.predict(new_password)

print("Password Strength:", prediction[0])
```

```
Password Strength: Strong
```